

9. Hafta - K-Nearest Neighbors (KNN) Modeli

May 6, 2025

1 9. Hafta: K-Nearest Neighbors (KNN) Modeli

6.1. KNN modelinin uygulanması ve hiperparametrelerinin açıklanması

6.2. Modelin eğitilmesi - Özellik ölçekleme yöntemlerinin uygulanması

6.3. Model sonuçlarının değerlendirilmesi

1.0.1 6.1. KNN modelinin uygulanması ve hiperparametrelerinin açıklanması

KNN (k nearest neighbors) algoritması, bir veri noktasını en yakın komşularının çoğunluğunun sınıfına göre sınıflandıran bir algoritmadır. Burada “en yakın” ifadesi, veri noktalarının arasındaki mesafeyi ifade eder, ve genellikle Öklid kullanılarak hesaplanır.

Verilen veri noktasının sınıfını belirlemek için K sayıda komşusuna bakar. K değeri, algoritmanın ne kadar esnek olduğunu belirler. Küçük k değerleri daha esnek hale getirirken, büyük K değerleri genelleştirmeyi artırır.

```
[121]: # Gerekli kütüphaneler
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_validate
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report)
```

```
[122]: # Veri setini oku
df = pd.read_csv("cleaned_df_data.csv")
```

```
[123]: # One-hot encoding işlemi
df = pd.get_dummies(df,
                    columns=["job", "marital", "education", "contact", "month",
                              "poutcome"],
                    drop_first=True)
```

```
[124]: # Sayısal değerlere dönüştürülecek sütunlar
# categorical_columns = ["job", "marital", "education", "contact", "month",
# ↪ "poutcome"]

# Her bir kategorik sütunu numaralandır ve aynı sütuna ata
# for col in categorical_columns:
#     df[col], _ = pd.factorize(df[col])

# Kontrol: İlk birkaç satırı yazdır
# print(df[categorical_columns].head())
```

```
[125]: # burada kaç veri var
df.shape
```

```
[125]: (45211, 41)
```

```
[126]: df
```

```
[126]:
```

	age	default	housing	loan	campaign	previous	y	balance_level	\
0	58	0	1	0	1	0	0	2	
1	44	0	1	0	1	0	0	1	
2	33	0	1	1	1	0	0	1	
3	47	0	1	0	1	0	0	1	
4	33	0	0	0	1	0	0	1	
...	...	...	...	...	...	...	...	...	
45206	51	0	0	0	3	0	1	1	
45207	71	0	0	0	2	0	1	1	
45208	72	0	0	0	5	3	1	2	
45209	57	0	0	0	4	0	0	1	
45210	37	0	0	0	2	11	0	2	

	has_any_credit	job_blue-collar	...	month_jul	month_jun	month_mar	\
0	1	0	...	0	0	0	
1	1	0	...	0	0	0	
2	1	0	...	0	0	0	
3	1	1	...	0	0	0	
4	0	0	...	0	0	0	
...	...	...	...	...	...	...	
45206	0	0	...	0	0	0	
45207	0	0	...	0	0	0	
45208	0	0	...	0	0	0	
45209	0	1	...	0	0	0	
45210	0	0	...	0	0	0	

	month_may	month_nov	month_oct	month_sep	poutcome_other	\
0	1	0	0	0	0	
1	1	0	0	0	0	

2	1	0	0	0	0
3	1	0	0	0	0
4	1	0	0	0	0
...	...	...	...	...	...
45206	0	1	0	0	0
45207	0	1	0	0	0
45208	0	1	0	0	0
45209	0	1	0	0	0
45210	0	1	0	0	1

	poutcome_success	poutcome_unknown
0	0	1
1	0	1
2	0	1
3	0	1
4	0	1
...	...	...
45206	0	1
45207	0	1
45208	1	0
45209	0	1
45210	0	0

[45211 rows x 41 columns]

```
[127]: df.isna().any().sum()
```

```
[127]: 0
```

```
[128]: df
```

```
[128]:
```

	age	default	housing	loan	campaign	previous	y	balance_level	\
0	58	0	1	0	1	0	0	2	
1	44	0	1	0	1	0	0	1	
2	33	0	1	1	1	0	0	1	
3	47	0	1	0	1	0	0	1	
4	33	0	0	0	1	0	0	1	
...	...	...	...	...	...	...	...	...	
45206	51	0	0	0	3	0	1	1	
45207	71	0	0	0	2	0	1	1	
45208	72	0	0	0	5	3	1	2	
45209	57	0	0	0	4	0	0	1	
45210	37	0	0	0	2	11	0	2	

	has_any_credit	job_blue-collar	...	month_jul	month_jun	month_mar	\
0	1	0	...	0	0	0	
1	1	0	...	0	0	0	

2	1	0	...	0	0	0
3	1	1	...	0	0	0
4	0	0	...	0	0	0
...	...	...	...	...	...	...
45206	0	0	...	0	0	0
45207	0	0	...	0	0	0
45208	0	0	...	0	0	0
45209	0	1	...	0	0	0
45210	0	0	...	0	0	0

	month_may	month_nov	month_oct	month_sep	poutcome_other	\
0	1	0	0	0	0	
1	1	0	0	0	0	
2	1	0	0	0	0	
3	1	0	0	0	0	
4	1	0	0	0	0	
...	...	...	...	...	...	
45206	0	1	0	0	0	
45207	0	1	0	0	0	
45208	0	1	0	0	0	
45209	0	1	0	0	0	
45210	0	1	0	0	1	

	poutcome_success	poutcome_unknown
0	0	1
1	0	1
2	0	1
3	0	1
4	0	1
...	...	...
45206	0	1
45207	0	1
45208	1	0
45209	0	1
45210	0	0

[45211 rows x 41 columns]

45211 veriden 40000 tanesi ile öğretilim, 5211 tanesi ile test edelim.

en sık yüzdelik ayırma olarak %70-%30 veya %75-%25 olarak

milyonlu satırlara çıktıkça %90'a yakın train olarak verilebilir.

```
[129]: # y sınıf oranlarını görelim
print(df["y"].value_counts(normalize=True))
```

0 0.883015

```
1    0.116985
Name: y, dtype: float64
```

Veri setimiz dengesiz bir yapıya sahip. Veri dengeleme yöntemlerinden weights='distance' parametresi ile bu durum incelenecektir.

6.2. Modelin eğitilmesi - Özellik ölçekleme yöntemlerinin uygulanması

```
[130]: # Bağımlı ve bağımsız değişkenleri ayır
y_knn = df["y"]
X_knn = df.drop("y", axis=1)
```

```
[131]: y_knn
```

```
[131]: 0      0
1      0
2      0
3      0
4      0
..
45206   1
45207   1
45208   1
45209   0
45210   0
Name: y, Length: 45211, dtype: int64
```

```
[132]: X_knn
```

```
[132]:
```

	age	default	housing	loan	campaign	previous	balance_level	\
0	58	0	1	0	1	0	2	
1	44	0	1	0	1	0	1	
2	33	0	1	1	1	0	1	
3	47	0	1	0	1	0	1	
4	33	0	0	0	1	0	1	
...	...	...	...	...	...	...	...	
45206	51	0	0	0	3	0	1	
45207	71	0	0	0	2	0	1	
45208	72	0	0	0	5	3	2	
45209	57	0	0	0	4	0	1	
45210	37	0	0	0	2	11	2	
...	...	...	...	...	...	...	...	
	has_any_credit	job_blue-collar	job_entrepreneur	...	month_jul	\		
0	1	0	0	...	0			
1	1	0	0	...	0			
2	1	0	1	...	0			
3	1	1	0	...	0			
4	0	0	0	...	0			
...	...	...	...	...	...			

45206	0	0	0	...	0
45207	0	0	0	...	0
45208	0	0	0	...	0
45209	0	1	0	...	0
45210	0	0	1	...	0

	month_jun	month_mar	month_may	month_nov	month_oct	month_sep	\
0	0	0	1	0	0	0	
1	0	0	1	0	0	0	
2	0	0	1	0	0	0	
3	0	0	1	0	0	0	
4	0	0	1	0	0	0	
...	...	...	...	...	...	...	
45206	0	0	0	1	0	0	
45207	0	0	0	1	0	0	
45208	0	0	0	1	0	0	
45209	0	0	0	1	0	0	
45210	0	0	0	1	0	0	

	poutcome_other	poutcome_success	poutcome_unknown
0	0	0	1
1	0	0	1
2	0	0	1
3	0	0	1
4	0	0	1
...	...	...	...
45206	0	0	1
45207	0	0	1
45208	0	1	0
45209	0	0	1
45210	1	0	0

[45211 rows x 40 columns]

```
[133]: # Veriyi eğitim ve test olarak ayır
X_train_knn, X_test_knn, y_train_knn, y_test_knn = train_test_split(X_knn,
↪y_knn, train_size=0.70, random_state=42)
```

StandardScaler, aşırı büyük veya küçük değerlerin etkisini azaltır.

Eğer modelleme için verilerin aynı ölçekte olması gerekiyorsa bu yöntem kullanılmalıdır.

```
[134]: # Standardizasyon işlemi
scaler = StandardScaler()
X_train_scaled_knn = scaler.fit_transform(X_train_knn)
X_test_scaled_knn = scaler.transform(X_test_knn)
```

En iyi k değerinden önce normal bir knn model kuralım.

```
[135]: # KNN modelini çağıralım.
```

```
knn_1 = KNeighborsClassifier(n_neighbors= 3)
```

```
[136]: # Modeli eğitiyoruz(öğrenme tamamlanacak)
```

```
knn_1.fit(X_train_scaled_knn, y_train_knn)
```

```
[136]: KNeighborsClassifier(n_neighbors=3)
```

```
[137]: # Test verisi üzerinde tahmin yap -- PREDICT : tahmin etmek
```

```
y_pred_knn_final = knn_1.predict(X_test_scaled_knn)
```

```
[138]: # Eğitilen modelin sonuçlarını hızlıca değerlendirelim
```

```
print(classification_report(y_test_knn, y_pred_knn_final))  
holdout = classification_report(y_test_knn, y_pred_knn_final)
```

	precision	recall	f1-score	support
0	0.91	0.96	0.93	11966
1	0.47	0.27	0.35	1598
accuracy			0.88	13564
macro avg	0.69	0.62	0.64	13564
weighted avg	0.86	0.88	0.86	13564

Modelimizin en iyi k değerini bulabilmesi için cross validation kullanalım ve en iyi k değerini bulalım.

```
[139]: # En iyi k değerini bulmak için doğrulama
```

```
k_values = np.arange(1, 21)
```

```
mean_test_score = []
```

```
for k in k_values:
```

```
    knn = KNeighborsClassifier(n_neighbors=k)
```

```
    cv_results = cross_validate(knn, X_train_scaled_knn, y_train_knn, cv=5,
```

```
    <scoring="accuracy")
```

```
    mean_test_score.append(cv_results["test_score"].mean())
```

```
[140]: # En iyi k değerini görselleştirme
```

```
plt.figure(figsize=(10, 5))
```

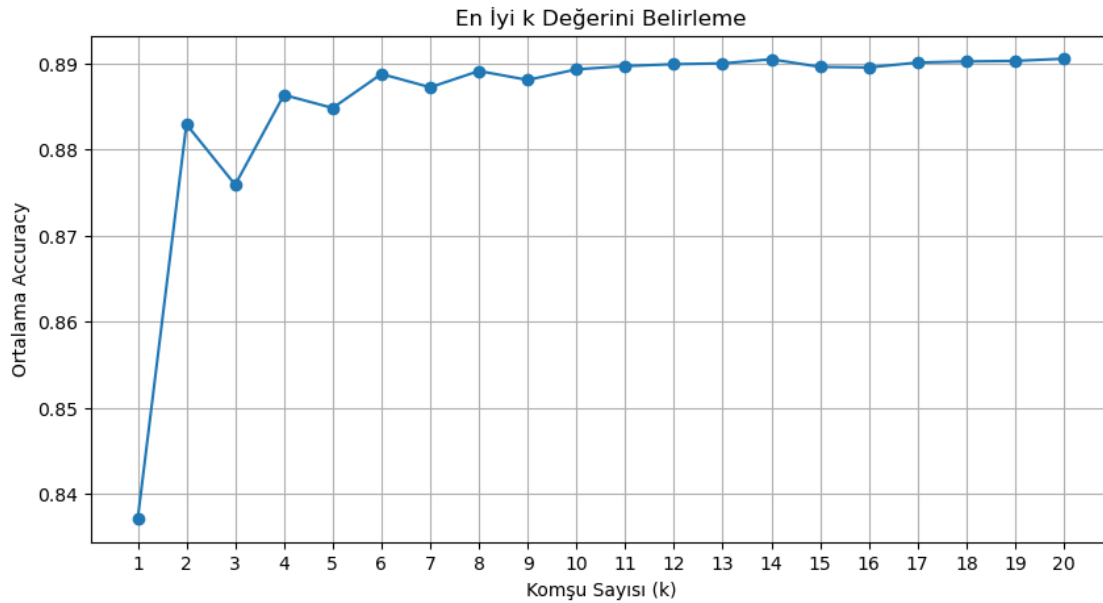
```
plt.plot(k_values, mean_test_score, marker="o")
```

```
plt.xlabel("Komşu Sayısı (k)")
```

```
plt.ylabel("Ortalama Accuracy")
```

```
plt.title("En İyi k Değerini Belirleme")
```

```
plt.grid(True)
plt.xticks(k_values)
plt.show()
```



```
[141]: # En iyi k değeriyle final model oluşturma

best_k = k_values[np.argmax(mean_test_score)]
print("En iyi k değeri:", best_k)

knn_final = KNeighborsClassifier(n_neighbors=best_k, weights='distance')
knn_final.fit(X_train_scaled_knn, y_train_knn)
```

En iyi k değeri: 20

```
[141]: KNeighborsClassifier(n_neighbors=20, weights='distance')
```

KNeighborsClassifier: - Bu, K-En Yakın Komşu (KNN) algoritması ile bir sınıflandırıcı oluşturur. - Bu model, tahmin yapılırken veri noktalarının en yakın k tane komşusuna bakar ve onların sınıfına göre karar verir.

weights='distance' Parametresi Ne İşe Yarar? Bu parametre KNN modelinin komşuları nasıl ağırlıklandıracağını belirler:

1. weights='uniform' (varsayılan): Bütün komşular eşit ağırlıktadır.

Yani yakın veya uzak olması fark etmez.

2. weights='distance'

Yakın komşulara daha fazla ağırlık verir.

Uzak komşuların etkisi azalır.

Bu, özellikle yoğun veri kümelerinde ve sınıfların sınırlarında daha iyi performans sağlar.

`weights='distance'` ayarı, KNN algoritmasında azınlık sınıfının tanınabilirliğini artırarak sınıflandırma performansını iyileştirebilir. Ancak, veri dengesizliğini gidermek için tek başına yeterli değildir. Bu nedenle, veri dengesizliği sorununu çözmek için ek yöntemlerle birlikte kullanılması önerilir.

Veri dengesizliğini gidermek için veri ön işleme teknikleri kullanılır: (Bu iki yönteme pdf içerisinde yer verilmiştir.)

- Oversampling → Örn: SMOTE
- Undersampling → Örn: RandomUnderSampler

```
[142]: # -----  
# TEST SETİ TAHMİNİ VE METRİKLER  
# -----  
y_pred_knn = knn_final.predict(X_test_scaled_knn)  
  
accuracy = accuracy_score(y_test_knn, y_pred_knn)  
precision = precision_score(y_test_knn, y_pred_knn)  
recall = recall_score(y_test_knn, y_pred_knn)  
f1 = f1_score(y_test_knn, y_pred_knn)  
  
print("\n--- TEST SETİ PERFORMANSI ---")  
print(f"Accuracy:  {accuracy:.2f}")  
print(f"Precision: {precision:.2f}")  
print(f"Recall:    {recall:.2f}")  
print(f"F1 Score:  {f1:.2f}")
```

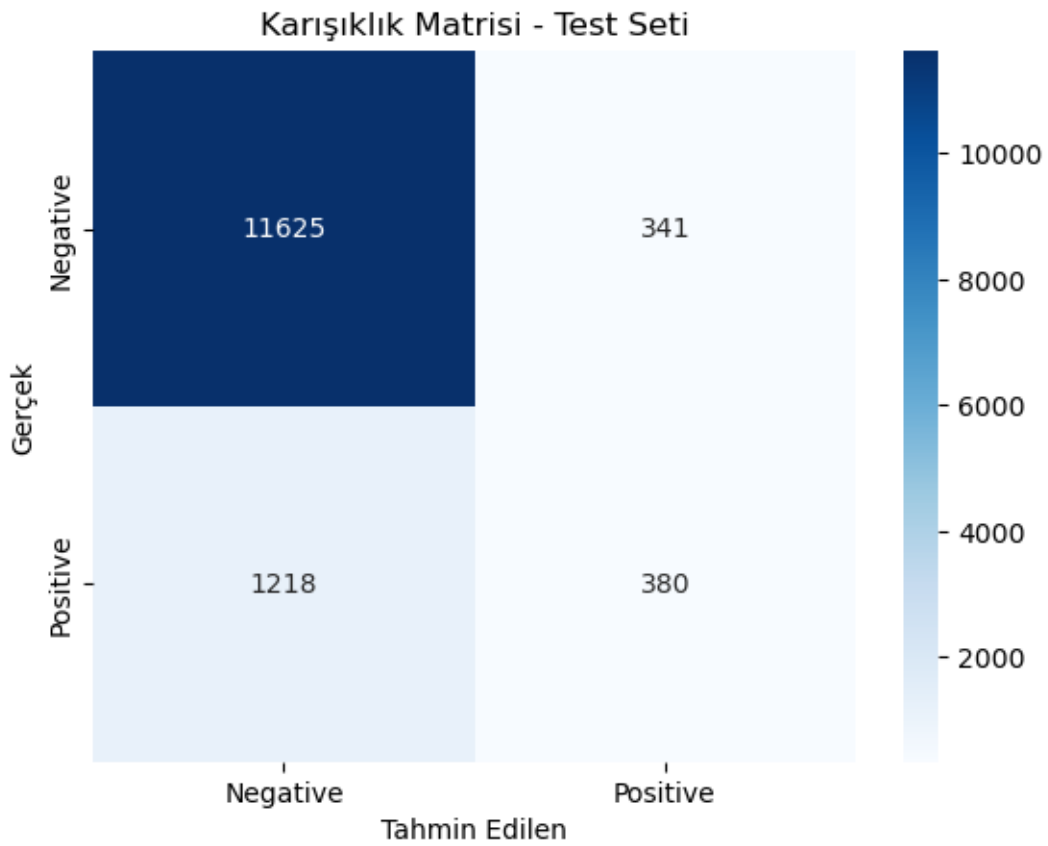
```
--- TEST SETİ PERFORMANSI ---  
Accuracy:  0.89  
Precision: 0.53  
Recall:    0.24  
F1 Score:  0.33
```

```
[143]: # Karışıklık Matrisi ve Sınıflandırma Raporu  
conf_matrix = confusion_matrix(y_test_knn, y_pred_knn)  
print("\nSınıflandırma Raporu:\n", classification_report(y_test_knn, y_pred_knn))  
  
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',  
            xticklabels=['Negative', 'Positive'],  
            yticklabels=['Negative', 'Positive'])  
plt.xlabel('Tahmin Edilen')  
plt.ylabel('Gerçek')  
plt.title('Karışıklık Matrisi - Test Seti')
```

```
plt.show()
```

Sınıflandırma Raporu:

	precision	recall	f1-score	support
0	0.91	0.97	0.94	11966
1	0.53	0.24	0.33	1598
accuracy			0.89	13564
macro avg	0.72	0.60	0.63	13564
weighted avg	0.86	0.89	0.87	13564



- 11625 → Gerçek negatif, doğru tahmin edilmiş (True Negative - TN)
 - Model, 11625 adet negatif sınıfı doğru bir şekilde “negatif” olarak tahmin etmiş.
- 341 → Gerçek negatif, yanlış pozitif tahmin edilmiş (False Positive - FP)
 - Model, 341 adet negatif sınıfı hatalı bir şekilde “pozitif” olarak tahmin etmiş.
- 1214 → Gerçek pozitif, yanlış negatif tahmin edilmiş (False Negative - FN)
 - Model, 1214 adet pozitif sınıfı yanlışlıkla “negatif” olarak tahmin etmiş.
- 380 → Gerçek pozitif, doğru tahmin edilmiş (True Positive - TP)

– Model, 380 adet pozitif sınıfı doğru bir şekilde “pozitif” olarak tahmin etmiş.

```
[144]: # -----  
# OVERFITTING KONTROLÜ (Eğitim Verisi)  
# -----  
y_train_pred_knn = knn_final.predict(X_train_scaled_knn)  
  
train_accuracy = accuracy_score(y_train_knn, y_train_pred_knn)  
train_precision = precision_score(y_train_knn, y_train_pred_knn)  
train_recall = recall_score(y_train_knn, y_train_pred_knn)  
train_f1 = f1_score(y_train_knn, y_train_pred_knn)  
  
print("\n--- EĞİTİM SETİ PERFORMANSI (Overfitting Kontrolü) ---")  
print(f"Train Accuracy : {train_accuracy:.2f}")  
print(f"Train Precision: {train_precision:.2f}")  
print(f"Train Recall   : {train_recall:.2f}")  
print(f"Train F1 Score  : {train_f1:.2f}")
```

```
--- EĞİTİM SETİ PERFORMANSI (Overfitting Kontrolü) ---  
Train Accuracy : 0.99  
Train Precision: 1.00  
Train Recall   : 0.92  
Train F1 Score : 0.96
```

1.0.2 Eğitim verisi üzerinde model neredeyse mükemmel çalışıyor. Bu kadar yüksek metrikler, modelin eğitildiği veriye çok iyi uydurulduğunu, hatta ezberlediğini gösteriyor. Bu, klasik bir overfitting belirtisidir.

1.0.3 Test Veri Seti'nde

- Recall çok düşük (0.24): Model pozitif sınıfları tanımakta zorlanıyor. Bu, özellikle pozitif sınıf önemliyse (örneğin, hasta olanlar, dolandırıcılık, terk etme durumu gibi) ciddi bir problemdir.
- F1 Skoru düşük (0.33): Precision ve recall'un birlikte dikkate alındığı bu metrik, dengesiz sınıflarda daha anlamlıdır. Düşük olması, modelin denge kurmakta zorlandığını gösteriyor.
- Accuracy'nin yüksek olması yanıltıcı: Çünkü veriniz büyük ihtimalle dengesiz. Model çoğunluk sınıfı (muhtemelen “negatif”) iyi tahmin ediyor ama azınlık sınıfı (“pozitif”) için başarısız.

2 Veri dengeleme tekniklerine göre sonuçlar

3 a. Oversampling (SMOTE) – Azınlık sınıftan daha fazla örnek üretmek

```
[145]: # Gerekli kütüphaneler
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, \
    accuracy_score, precision_score, recall_score, f1_score
import seaborn as sns
import matplotlib.pyplot as plt
```

```
[146]: # 1. Sadece eğitim verisi üzerinden SMOTE uygulama
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_train_scaled_knn, y_train_knn)
```

```
[147]: # 2. Modeli eğit
knn = KNeighborsClassifier(n_neighbors=20, weights='distance')
knn.fit(X_resampled, y_resampled)
```

```
[147]: KNeighborsClassifier(n_neighbors=20, weights='distance')
```

```
[148]: # 3. Test verisi üzerinde tahmin yap (ölçeklenmiş orijinal test verisi,
    ↪ kullanılmalı!)
y_pred = knn.predict(X_test_scaled_knn)
```

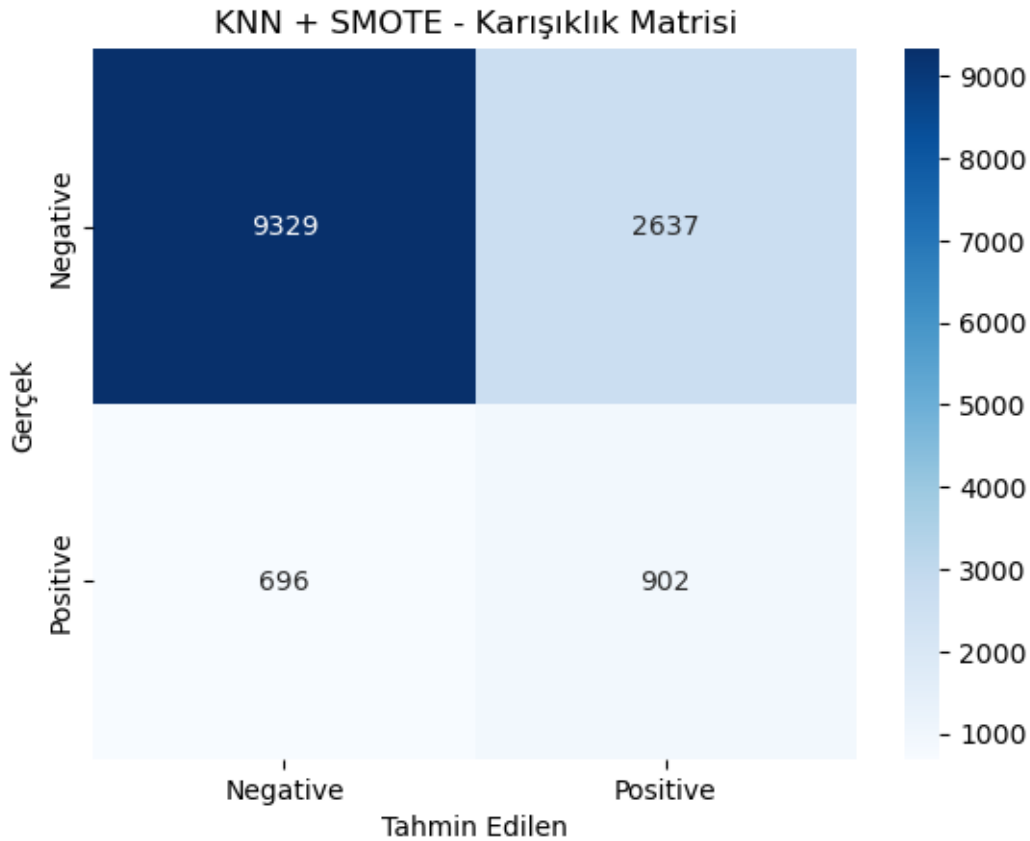
```
[149]: # 4. Test performansı
print("TEST SETİ PERFORMANSI:\n")
print(confusion_matrix(y_test_knn, y_pred))
print(classification_report(y_test_knn, y_pred))
```

TEST SETİ PERFORMANSI:

```
[[9329 2637]
 [ 696  902]]
```

	precision	recall	f1-score	support
0	0.93	0.78	0.85	11966
1	0.25	0.56	0.35	1598
accuracy			0.75	13564
macro avg	0.59	0.67	0.60	13564
weighted avg	0.85	0.75	0.79	13564

```
[150]: # 5. Karışıklık Matrisi görselleştirme
cm = confusion_matrix(y_test_knn, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Negative', 'Positive'],
            yticklabels=['Negative', 'Positive'])
plt.xlabel('Tahmin Edilen')
plt.ylabel('Gerçek')
plt.title('KNN + SMOTE - Karışıklık Matrisi')
plt.show()
```



- 9329 → Gerçek negatif, doğru tahmin edilmiş (True Negative - TN)
 - Model, 9329 adet negatif sınıfı doğru bir şekilde “negatif” olarak tahmin etmiş.
- 2637 → Gerçek negatif, yanlış pozitif tahmin edilmiş (False Positive - FP)
 - Model, 2637 adet negatif sınıfı hatalı bir şekilde “pozitif” olarak tahmin etmiş.
- 696 → Gerçek pozitif, yanlış negatif tahmin edilmiş (False Negative - FN)
 - Model, 696 adet pozitif sınıfı yanlışlıkla “negatif” olarak tahmin etmiş.
- 902 → Gerçek pozitif, doğru tahmin edilmiş (True Positive - TP)
 - Model, 902 adet pozitif sınıfı doğru bir şekilde “pozitif” olarak tahmin etmiş.

```
[151]: # 6. Performans metrikleri
accuracy = accuracy_score(y_test_knn, y_pred)
precision = precision_score(y_test_knn, y_pred)
recall = recall_score(y_test_knn, y_pred)
f1 = f1_score(y_test_knn, y_pred)

print("TEST SETİ METRİKLERİ:")
print(f"Accuracy : {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall    : {recall:.2f}")
print(f"F1 Score  : {f1:.2f}")
```

```
TEST SETİ METRİKLERİ:
Accuracy : 0.75
Precision: 0.25
Recall    : 0.56
F1 Score  : 0.35
```

```
[152]: # 7. Eğitim seti üzerinde tahmin (Aşırı öğrenme kontrolü)
y_train_pred = knn.predict(X_resampled)

train_accuracy = accuracy_score(y_resampled, y_train_pred)
train_precision = precision_score(y_resampled, y_train_pred)
train_recall = recall_score(y_resampled, y_train_pred)
train_f1 = f1_score(y_resampled, y_train_pred)

print("\nEĞİTİM SETİ PERFORMANSI (Overfitting Kontrolü):")
print(f"Train Accuracy : {train_accuracy:.2f}")
print(f"Train Precision: {train_precision:.2f}")
print(f"Train Recall    : {train_recall:.2f}")
print(f"Train F1 Score  : {train_f1:.2f}")
```

```
EĞİTİM SETİ PERFORMANSI (Overfitting Kontrolü):
Train Accuracy : 0.99
Train Precision: 1.00
Train Recall    : 0.99
Train F1 Score  : 0.99
```

SMOTE, Recall'ı iyileştirerek olumlu katkı sağlamış, bu önemli çünkü dengesiz sınıflarda genelde düşük olan sınıfı yakalamak zordur. Test performansı düşük olduğu için model genelleme yeteneğini kaybetmiş durumdadır.

- Eğitim seti yine çok yüksek: aşırı öğrenme hâlâ mevcut.
- Ancak Recall test setinde yükselmiş (0.56): bu, daha fazla pozitif sınıfı doğru tahmin etmeye başladığını gösteriyor.
- Fakat Precision düştü (0.25), yani yanlış pozitifler arttı. Bu da SMOTE'un etkisiyle modelin biraz fazla "pozitif" sınıf tahmin ettiğini gösterir.

- F1 skoru küçük bir artış göstermiş ama hâlâ düşük.

4 b. Undersampling – Çoğunluk sınıftan örnek sayısını azaltmak

```
[153]: # Gerekli kütüphaneler
from imblearn.under_sampling import RandomUnderSampler
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (confusion_matrix, classification_report,
                             accuracy_score, precision_score, recall_score,
                             ↪f1_score)
import matplotlib.pyplot as plt
import seaborn as sns

[154]: # 1. Veriyi eğitim ve test olarak ayır (önceden ölçeklenmiş veriler kullanıldı,
       ↪varsayılıyor)
       # Örneğin: X_train_scaled_knn, X_test_scaled_knn, y_train_knn, y_test_knn

[155]: # 2. RandomUnderSampler sadece eğitim verisine uygulanır
rus = RandomUnderSampler(random_state=42)
X_train_resampled, y_train_resampled = rus.fit_resample(X_train_scaled_knn,
       ↪y_train_knn)

[156]: # 3. KNN modelini eğit
knn = KNeighborsClassifier(n_neighbors=20, weights='distance')
knn.fit(X_train_resampled, y_train_resampled)

[156]: KNeighborsClassifier(n_neighbors=20, weights='distance')

[157]: # 4. Test seti tahmini
y_test_pred = knn.predict(X_test_scaled_knn)

[158]: # 5. Performans metrikleri (TEST)
test_acc = accuracy_score(y_test_knn, y_test_pred)
test_prec = precision_score(y_test_knn, y_test_pred)
test_rec = recall_score(y_test_knn, y_test_pred)
test_f1 = f1_score(y_test_knn, y_test_pred)

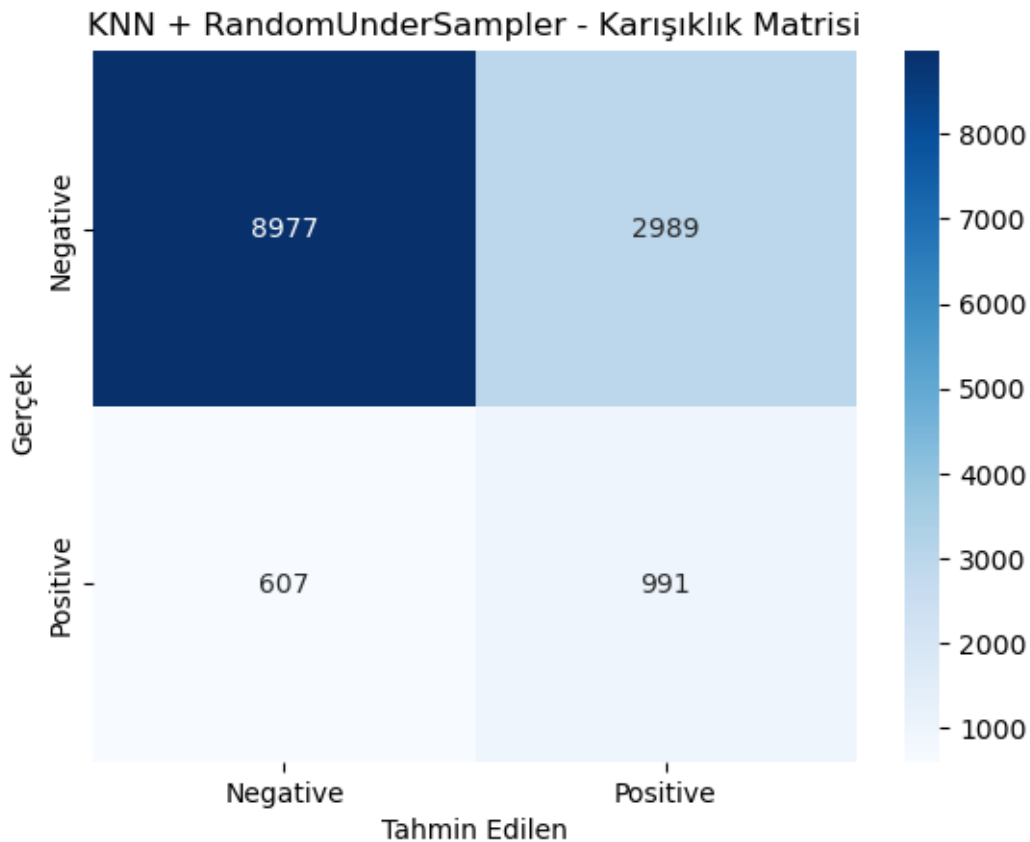
print("\nTEST SETİ PERFORMANSI:")
print(f"Test Accuracy : {test_acc:.2f}")
print(f"Test Precision: {test_prec:.2f}")
print(f"Test Recall   : {test_rec:.2f}")
print(f"Test F1-Score  : {test_f1:.2f}")
```

TEST SETİ PERFORMANSI:
Test Accuracy : 0.73

Test Precision: 0.25
Test Recall : 0.62
Test F1-Score : 0.36

```
[159]: # 6. Karışıklık Matrisi ve rapor
cm = confusion_matrix(y_test_knn, y_test_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Negative', 'Positive'],
            yticklabels=['Negative', 'Positive'])
plt.xlabel('Tahmin Edilen')
plt.ylabel('Gerçek')
plt.title('KNN + RandomUnderSampler - Karışıklık Matrisi')
plt.show()

print("\nClassification Report:")
print(classification_report(y_test_knn, y_test_pred))
```



Classification Report:

precision	recall	f1-score	support
-----------	--------	----------	---------

0	0.94	0.75	0.83	11966
1	0.25	0.62	0.36	1598
accuracy			0.73	13564
macro avg	0.59	0.69	0.59	13564
weighted avg	0.86	0.73	0.78	13564

- 8977 → Gerçek negatif, doğru tahmin edilmiş (True Negative - TN)
 - Model, 8977 adet negatif sınıfı doğru bir şekilde “negatif” olarak tahmin etmiş.
- 2989 → Gerçek negatif, yanlış pozitif tahmin edilmiş (False Positive - FP)
 - Model, 2989 adet negatif sınıfı hatalı bir şekilde “pozitif” olarak tahmin etmiş.
- 607 → Gerçek pozitif, yanlış negatif tahmin edilmiş (False Negative - FN)
 - Model, 607 adet pozitif sınıfı yanlışlıkla “negatif” olarak tahmin etmiş.
- 991 → Gerçek pozitif, doğru tahmin edilmiş (True Positive - TP)
 - Model, 991 adet pozitif sınıfı doğru bir şekilde “pozitif” olarak tahmin etmiş.

```
[160]: # 7. Eğitim seti tahmini (Overfitting kontrolü)
y_train_pred = knn.predict(X_train_resampled)

train_acc = accuracy_score(y_train_resampled, y_train_pred)
train_prec = precision_score(y_train_resampled, y_train_pred)
train_rec = recall_score(y_train_resampled, y_train_pred)
train_f1 = f1_score(y_train_resampled, y_train_pred)

print("\nEĞİTİM SETİ PERFORMANSI (Overfitting Kontrolü):")
print(f"Train Accuracy : {train_acc:.2f}")
print(f"Train Precision: {train_prec:.2f}")
print(f"Train Recall    : {train_rec:.2f}")
print(f"Train F1-Score  : {train_f1:.2f}")
```

EĞİTİM SETİ PERFORMANSI (Overfitting Kontrolü):

Train Accuracy : 0.99
 Train Precision: 1.00
 Train Recall : 0.99
 Train F1-Score : 0.99

- Pozitif Noktalar:
 - Recall (0.62) SMOTE’tan da yüksek, yani model artık pozitif sınıfı daha iyi yakalıyor.
 - Precision (0.25) düşük ama bu, sınıf dengesizliğinde beklenen bir durumdur. Yanlış pozitiflerin sayısı fazla.
 - F1 Skoru (0.36), önceki SMOTE sonucuyla benzer, ancak recall’un daha yüksek olması avantaj sağlar.
- Negatif Noktalar:
 - Train ve Test performansı arasındaki fark hâlâ büyük: Eğitim setinde neredeyse %100 başarı, testte ise ortalama. Bu hâlâ aşırı öğrenme (overfitting) belirtisidir.
 - Undersampling, veri miktarını düşürdüğü için modelin genelleme yeteneği sınırlı olabilir. Ama yine de pozitif sınıfı bulma konusunda iyi iş çıkarmış.

Burada KNN ile kurulan modelde aşırı öğrenme değerleri veri dengeleme tekniklerinde dahi çok yüksek çıkmıştır. O sebeple başka bir algoritme tekniği kullanılması daha uygun bulunmuştur.